8 EXTENDING SOURCE-DPOR AND OPTIMAL-DPOR TO SUPPORT BLOCKING

In this section, we consider how to extend the techniques of the previous sections for the general situation in which processes can disable each other. In other words, we will no longer make use of Assumption 3.1. Processes can disable each other via shared synchronization objects, such as locks, condition variables, etc. It is also possible for processes to disable each other via less visible mechanisms; one such is to use `await` statements, by means of which any update to a shared variable can disable an `await` statement that depends on this shared variable. When extending our techniques to the case in which processes can disable each other, it turns out that the general case of `await` statements requires a more sophisticated modification than the case of locks. We will therefore first consider how to extend Algorithms 1 and 2 to handle locks, and thereafter consider the general case.

8.1 Handling Locks

Let us extend Algorithms 1 and 2 to the case that disabling between processes happens only via locks. We assume a set of locks that are disjoint from ordinary memory locations and only used in lock operations. Each lock l can take values 0 or 1 (initial value) and supports the operations `lock(`l`)` and `unlock(`l`)`, where `lock(`l`)` decrements l from 1 to 0 if l is 1 and blocks if l is 0, and where `unlock(`l`)` assigns 1 to l (and never blocks). The only operation that can disable another process is `lock(`l`)`, which disables any other operation of form `lock(`l`)`. We assume that the happens-before relation relates a `lock(`l`)` operation with any other operation of form `lock(`l`)` or `unlock(`l`)` on the same lock, that the `lock(`l`)` operation is independent of other operations, and that two `unlock(`l`)` operations by different threads are independent. Note that we do not assume that programs use locks in a "well-structured" way or that the program is data-race free.

8.1.1 Extending Source-DPOR to Handle Locks. To handle locks, the basic modification to Algorithm 1 is to extend the concept of races. An important part of Algorithm 1 (at line 5) is to detect a race between two events e and $next_{[E]}(p)$. Let us denote $next_{[E]}(p)$ by e'. In the absence of locks, whenever e is in a race with e', and e is performed before e', then e' must appear sometime later in the execution since it cannot be disabled by e. This implies that the race detection mechanism of line 5 in Algorithm 1 is sufficient for detecting any race. However, in the presence of locks, we must detect races also in the case where e' has been disabled by e. The needed extension is to consider the case where e' performs a `lock()` operation, regardless of whether $\widehat{e'}$ is blocked or not, and to investigate alternative executions in which e' is performed before the most recently performed `lock()` operation. Such an extension should work when $\widehat{e'}$ is blocked, since it is necessary to explore the case where e' occurs before the most recently performed `lock()` operation, when it is not blocked. Additionally, it should *also* work when $\widehat{e'}$ is not blocked. The reason is that Algorithm 1 will only find the "race" between the enabled lock event e' and the previous `unlock()` operation: obviously these two events cannot be reversed. Instead it is necessary to consider the race between e' and the most recently performed `lock()` operation; these events can be performed in reverse order.

Our extension of Algorithm 1 to programs with locks is shown in Algorithm 3. The extension consists in adding an extra case to the race detection phase, found at lines 2–8. This extra case considers all processes p that are about to perform a `lock()` operation on some lock l, regardless of whether they are blocked or not. For each such process, we find the preceding `lock(`l`)` operation in E, since that operation is in a race with the next step of process p, according to our extended definition of "race". Note that the next step of process p, which is a `lock(`l`)` operation, can be either enabled or disabled. Thereafter, the race detection proceeds analogously as in the non-blocking